Skills Network

Extracting and Visualizing Stock Data

Description

Extracting essential data from a dataset and displaying it is a necessary part of data science; therefore individuals can make correct decisions based on the data. In this assignment, you will extract some stock data, you will then display this data in a graph.

Table of Contents

- Define a Function that Makes a Graph
- Question 1: Use yfinance to Extract Stock Data
- Question 2: Use Webscraping to Extract Tesla Revenue Data
- Question 3: Use yfinance to Extract Stock Data
- Question 4: Use Webscraping to Extract GME Revenue Data
- Question 5: Plot Tesla Stock Graph
- Question 6: Plot GameStop Stock Graph

Estimated Time Needed: **30 min**

Note:- If you are working Locally using anaconda, please uncomment the following code and execute it. Use the version as per your python version.

```
!pip install yfinance
!pip install bs4
!pip install nbformat
!pip install --upgrade plotly
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: yfinance in c:\users\user\appdata\roaming\python\python311\site-packages (0.2.65)
Requirement already satisfied: pandas>=1.3.0 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (2.2.
Requirement already satisfied: numpy>=1.16.5 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (2.2.
Requirement already satisfied: requests>=2.31 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (2.3
Requirement already satisfied: multitasking>=0.0.7 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance)
Requirement already satisfied: platformdirs>=2.0.0 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance)
Requirement already satisfied: pytz>=2022.5 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (2025.
Requirement already satisfied: frozendict>=2.3.4 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (
Requirement already satisfied: peewee>=3.16.2 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (3.1
Requirement already satisfied: beautifulsoup4>=4.11.1 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinan
Requirement already satisfied: curl_cffi>=0.7 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (0.1
Requirement already satisfied: protobuf>=3.19.0 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (6
Requirement already satisfied: websockets>=13.0 in c:\users\user\appdata\roaming\python\python311\site-packages (from yfinance) (1
Requirement already satisfied: soupsieve>1.2 in c:\users\user\appdata\roaming\python\python311\site-packages (from beautifulsoup4>
Requirement already satisfied: cffi>=1.12.0 in c:\users\user\appdata\roaming\python\python311\site-packages (from curl_cffi>=0.7->
Requirement already satisfied: certifi>=2024.2.2 in c:\users\user\appdata\roaming\python\python311\site-packages (from curl_cffi>=
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\user\appdata\roaming\python\python311\site-packages (from pandas
Requirement already satisfied: tzdata>=2022.7 in c:\users\user\appdata\roaming\python\python311\site-packages (from pandas>=1.3.0-
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\user\appdata\roaming\python\python311\site-packages (from requ
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\appdata\roaming\python\python311\site-packages (from requests>=2.31->
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\user\appdata\roaming\python\python311\site-packages (from requests>=
Requirement already satisfied: pycparser in c:\users\user\appdata\roaming\python\python311\site-packages (from cffi>=1.12.0->curl_
Requirement already satisfied: six>=1.5 in c:\users\user\appdata\roaming\python\python311\site-packages (from python-dateutil>=2.8

[notice] A new release of pip is available: 24.3.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: bs4 in c:\users\user\appdata\roaming\python\python311\site-packages (0.0.2)
Requirement already satisfied: beautifulsoup4 in c:\users\user\appdata\roaming\python\python311\site-packages (from bs4) (4.12.3)
Requirement already satisfied: soupsieve>1.2 in c:\users\user\appdata\roaming\python\python311\site-packages (from beautifulsoup4-

[notice] A new release of pip is available: 24.3.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
[notice] A new release of pip is available: 24.3.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip

Requirement already satisfied: nbformat in c:\users\user\appdata\roaming\python\python311\site-packages (5.10.4)
Requirement already satisfied: fastjsonschema>=2.15 in c:\users\user\appdata\roaming\python\python311\site-packages (from nbformat
Requirement already satisfied: jsonschema>=2.6 in c:\users\user\appdata\roaming\python\python311\site-packages (from nbformat) (4.
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in c:\users\user\appdata\roaming\python\python311\site-packages (from nb
```

```
Requirement already satisfied: traitlets>=5.1 in c:\users\user\appdata\roaming\python\python311\site-packages (from nbformat) (5.1
Requirement already satisfied: attrs>=22.2.0 in c:\users\user\appdata\roaming\python\python311\site-packages (from jsonschema>=2.6
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\users\user\appdata\roaming\python\python311\site-package
Requirement already satisfied: referencing>=0.28.4 in c:\users\user\appdata\roaming\python\python311\site-packages (from jsonschem
Requirement already satisfied: rpds-py>=0.7.1 in c:\users\user\appdata\roaming\python\python311\site-packages (from jsonschema>=2.
Requirement already satisfied: platformdirs>=2.5 in c:\users\user\appdata\roaming\python\python311\site-packages (from jupyter-cor
Requirement already satisfied: pywin32>=300 in c:\users\user\appdata\roaming\python\python311\site-packages (from jupyter-core!=5.
Requirement already satisfied: typing-extensions>=4.4.0 in c:\users\user\appdata\roaming\python\python311\site-packages (from refe
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: plotly in c:\users\user\appdata\roaming\python\python311\site-packages (6.3.0)
Requirement already satisfied: narwhals>=1.15.1 in c:\users\user\appdata\roaming\python\python311\site-packages (from plotly) (2.2
Requirement already satisfied: packaging in c:\users\user\appdata\roaming\python\python311\site-packages (from plotly) (24.2)

[notice] A new release of pip is available: 24.3.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
import yfinance as yf
import pandas as pd
import requests
from bs4 import BeautifulSoup
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

```
import plotly.io as pio
pio.renderers.default = "iframe"
```

In Python, you can ignore warnings using the warnings module. You can use the filterwarnings function to filter or ignore specific warning messages or categories.

```
import warnings
# Ignore all warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

Define Graphing Function

In this section, we define the function `make_graph`. **You don't have to know how the function works, you should only care about the inputs. It takes a dataframe with stock data (dataframe must contain Date and Close columns), a dataframe with revenue data (dataframe must contain Date and Revenue columns), and the name of the stock.**

```
def make_graph(stock_data, revenue_data, stock):
    fig = make_subplots(rows=2, cols=1, shared_xaxes=True, subplot_titles=("Historical Share Price", "Historical Revenue"), vertical_spa
    stock_data_specific = stock_data[stock_data.Date <= '2021-06-14']
    revenue_data_specific = revenue_data[revenue_data.Date <= '2021-04-30']
    fig.add_trace(go.Scatter(x=pd.to_datetime(stock_data_specific.Date, infer_datetime_format=True), y=stock_data_specific.Close.astype(
    fig.add_trace(go.Scatter(x=pd.to_datetime(revenue_data_specific.Date, infer_datetime_format=True), y=revenue_data_specific.Revenue.a
    fig.update_xaxes(title_text="Date", row=1, col=1)
    fig.update_xaxes(title_text="Date", row=2, col=1)
    fig.update_yaxes(title_text="Price ($US)", row=1, col=1)
    fig.update_yaxes(title_text="Revenue ($US Millions)", row=2, col=1)
    fig.update_layout(showlegend=False,
    height=900,
    title=stock,
    xaxis_rangeslider_visible=True)
    fig.show()
    from IPython.display import display, HTML
    fig_html = fig.to_html()
    display(HTML(fig_html))
```

Use the make_graph function that we've already defined. You'll need to invoke it in questions 5 and 6 to display the graphs and create the dashboard.

> **Note: You don't need to redefine the function for plotting graphs anywhere else in this notebook; just use the existing function.**

Question 1: Use yfinance to Extract Stock Data

Using the `Ticker` function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is Tesla and its ticker symbol is `TSLA`.

```
import yfinance as yf
ticker=yf.Ticker('TSLA')
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `tesla_data`. Set the `period` parameter to `"max"` so we get information for the maximum amount of time.

```
tesla_data=ticker.history(period="max")
```

Reset the index using the `reset_index(inplace=True)` function on the tesla_data DataFrame and display the first five rows of the `tesla_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 1 to the results below.

```
tesla_data.reset_index(inplace=True)
print(tesla_data.head())
```

```
   index                      Date      Open      High       Low     Close  \
0      0 2010-06-29 00:00:00-04:00  1.266667  1.666667  1.169333  1.592667
1      1 2010-06-30 00:00:00-04:00  1.719333  2.028000  1.553333  1.588667
2      2 2010-07-01 00:00:00-04:00  1.666667  1.728000  1.351333  1.464000
3      3 2010-07-02 00:00:00-04:00  1.533333  1.540000  1.247333  1.280000
4      4 2010-07-06 00:00:00-04:00  1.333333  1.333333  1.055333  1.074000

       Volume  Dividends  Stock Splits
0  281494500        0.0           0.0
1  257806500        0.0           0.0
2  123282000        0.0           0.0
3   77097000        0.0           0.0
4  103003500        0.0           0.0
```

Question 2: Use Webscraping to Extract Tesla Revenue Data

Use the `requests` library to download the webpage https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/revenue.htm Save the text of the response as a variable named `html_data`.

```
url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/rev
response=requests.get(url)
html_data=response.text
```

Parse the html data using `beautiful_soup` using parser i.e `html5lib` or `html.parser`.

```
soup = BeautifulSoup(html_data, "html.parser")
```

Using `BeautifulSoup` or the `read_html` function extract the table with `Tesla Revenue` and store it into a dataframe named `tesla_revenue`. The dataframe should have columns `Date` and `Revenue`.

▶ Step-by-step instructions

▶ Click here if you need help locating the table

```
import pandas as pd
from bs4 import BeautifulSoup
tesla_revenue = pd.DataFrame(columns=["Date", "Revenue"])
table = soup.find_all("tbody")[1]   # Tesla Quarterly Revenue table
for row in table.find_all("tr"):
    cols = row.find_all("td")


    if len(cols) == 2:
        date = cols[0].text.strip()
        revenue = cols[1].text.strip()


        tesla_revenue = pd.concat([tesla_revenue, pd.DataFrame({"Date":[date], "Revenue":[revenue]})],
                                  ignore_index=True)

# 6. Clean up the Revenue column (remove $ and commas, drop NAs)
tesla_revenue["Revenue"] = tesla_revenue["Revenue"].str.replace(r"\$|,", "", regex=True)
tesla_revenue.dropna(inplace=True)
```

Execute the following line to remove the comma and dollar sign from the `Revenue` column.

```
tesla_revenue["Revenue"] = tesla_revenue['Revenue'].str.replace(',|\$',"",regex=True)
```

Execute the following lines to remove an null or empty strings in the Revenue column.

```
tesla_revenue.dropna(inplace=True)

tesla_revenue = tesla_revenue[tesla_revenue['Revenue'] != ""]
```

Display the last 5 row of the `tesla_revenue` dataframe using the `tail` function. Take a screenshot of the results.

```
print(tesla_revenue.tail())
```

```
           Date Revenue
48  2010-09-30      31
49  2010-06-30      28
50  2010-03-31      21
52  2009-09-30      46
53  2009-06-30      27
```

Question 3: Use yfinance to Extract Stock Data

Using the `Ticker` function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is GameStop and its ticker symbol is `GME`.

```
ticker = yf.Ticker("GME")
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `gme_data`. Set the `period` parameter to `"max"` so we get information for the maximum amount of time.

```
gme_data = ticker.history(period="max")
```

Reset the index using the `reset_index(inplace=True)` function on the gme_data DataFrame and display the first five rows of the `gme_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 3 to the results below.

```
gme_data.reset_index(inplace=True)
print(gme_data.head())
```

```
                       Date      Open      High       Low     Close    Volume  \
0 2002-02-13 00:00:00-05:00  1.620128  1.693350  1.603296  1.691666  76216000
1 2002-02-14 00:00:00-05:00  1.712708  1.716074  1.670626  1.683251  11021600
2 2002-02-15 00:00:00-05:00  1.683250  1.687458  1.658002  1.674834   8389600
3 2002-02-19 00:00:00-05:00  1.666418  1.666418  1.578047  1.607504   7410400
4 2002-02-20 00:00:00-05:00  1.615920  1.662209  1.603296  1.662209   6892800

   Dividends  Stock Splits
0        0.0           0.0
1        0.0           0.0
2        0.0           0.0
3        0.0           0.0
4        0.0           0.0
```

Question 4: Use Webscraping to Extract GME Revenue Data

Use the `requests` library to download the webpage https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/stock.html. Save the text of the response as a variable named `html_data_2`.

```
url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/st
response = requests.get(url)
html_data_2 = response.text
```

Parse the html data using `beautiful_soup` using parser i.e `html5lib` or `html.parser`.

```
soup2 = BeautifulSoup(html_data_2, "html.parser")
```

Using `BeautifulSoup` or the `read_html` function extract the table with `GameStop Revenue` and store it into a dataframe named `gme_revenue`. The dataframe should have columns `Date` and `Revenue`. Make sure the comma and dollar sign is removed from the `Revenue` column.

> **Note: Use the method similar to what you did in question 2.**

▶ Click here if you need help locating the table

```
# 1. Create an empty DataFrame
gme_revenue = pd.DataFrame(columns=["Date", "Revenue"])

# 2. Locate the correct table (second table in HTML)
table = soup2.find_all("tbody")[1]

# 3. Iterate through rows
for row in table.find_all("tr"):
    cols = row.find_all("td")
    if len(cols) == 2:
        date = cols[0].text.strip()
        revenue = cols[1].text.strip()
        gme_revenue = pd.concat([gme_revenue,
                                 pd.DataFrame({"Date":[date], "Revenue":[revenue]})],
                                ignore_index=True)

# 4. Clean Revenue column
gme_revenue["Revenue"] = gme_revenue["Revenue"].str.replace(r"\$|,", "", regex=True)
gme_revenue.dropna(inplace=True)
```

Display the last five rows of the `gme_revenue` dataframe using the `tail` function. Take a screenshot of the results.

```
print(gme_revenue.tail())
```

```
           Date Revenue
57  2006-01-31    1667
58  2005-10-31     534
59  2005-07-31     416
60  2005-04-30     475
61  2005-01-31     709
```

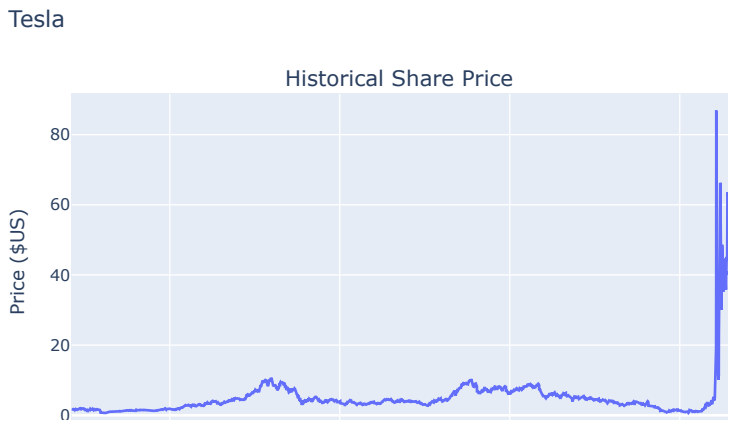
Question 5: Plot Tesla Stock Graph

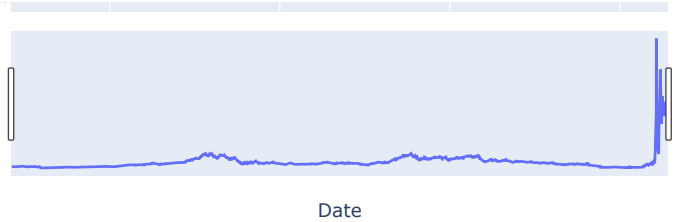
Use the `make_graph` function to graph the Tesla Stock Data, also provide a title for the graph. Note the graph will only show data upto June 2021.

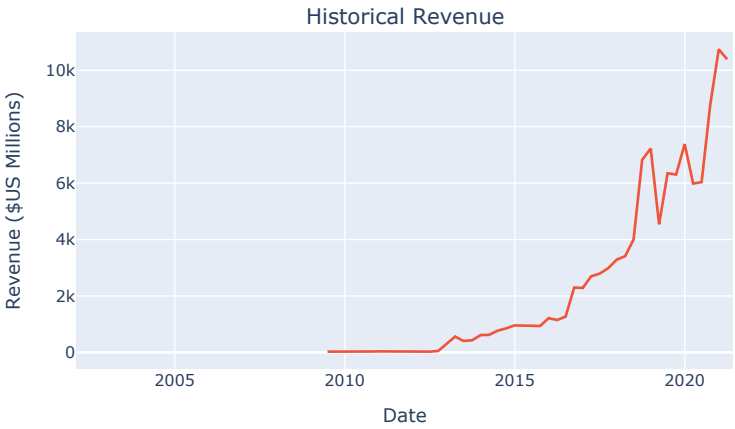
▶ Hint

```
tesla_revenue["Revenue"] = tesla_revenue["Revenue"].str.replace(r"\$|,", "", regex=True)

# Replace empty strings with NaN
tesla_revenue["Revenue"].replace("", float("nan"), inplace=True)

# Drop rows where Revenue is NaN
tesla_revenue.dropna(subset=["Revenue"], inplace=True)

# Convert Revenue to float
tesla_revenue["Revenue"] = tesla_revenue["Revenue"].astype(float)

# Ensure Date is datetime
tesla_revenue["Date"] = pd.to_datetime(tesla_revenue["Date"])
tesla_data["Date"] = pd.to_datetime(tesla_data["Date"])

# Filter data up to June 2021
tesla_data_filtered = tesla_data[tesla_data["Date"] <= "2021-06-01"]
tesla_revenue_filtered = tesla_revenue[tesla_revenue["Date"] <= "2021-06-01"]

# Call the provided make_graph function
make_graph(tesla_data_filtered, tesla_revenue_filtered, "Tesla")
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_11812\3568987243.py:4: FutureWarning:

A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[co]


C:\Users\User\AppData\Local\Temp\ipykernel_11812\109047474.py:5: UserWarning:

The argument 'infer_datetime_format' is deprecated and will be removed in a future version. A strict version of it is now the defau]

C:\Users\User\AppData\Local\Temp\ipykernel_11812\109047474.py:6: UserWarning:

The argument 'infer_datetime_format' is deprecated and will be removed in a future version. A strict version of it is now the defau]
```







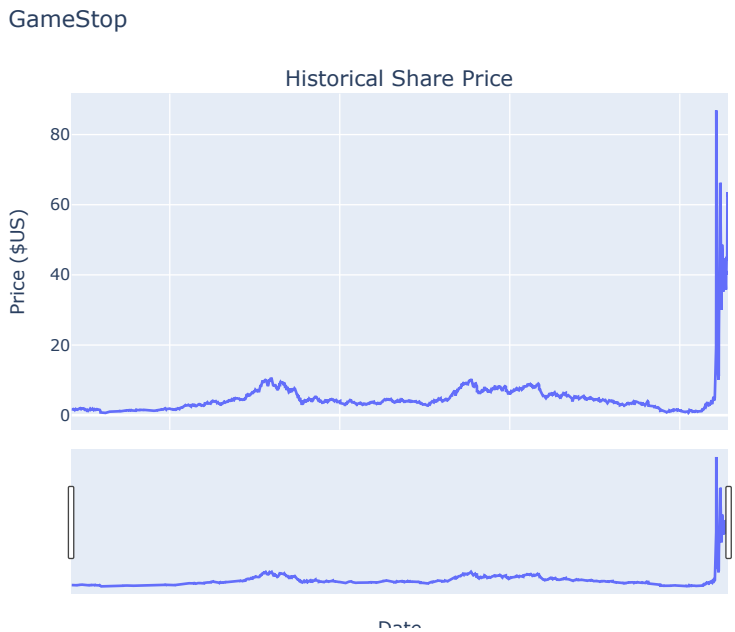
Question 6: Plot GameStop Stock Graph

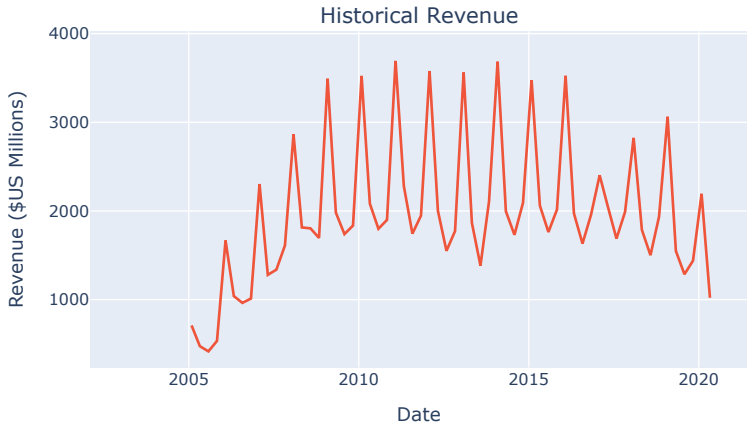
Use the `make_graph` function to graph the GameStop Stock Data, also provide a title for the graph. The structure to call the `make_graph` function is `make_graph(gme_data, gme_revenue, 'GameStop')`. Note the graph will only show data upto June 2021.

▶ Hint

```
gme_revenue["Revenue"] = gme_revenue["Revenue"].str.replace(r"\$|,", "", regex=True)
gme_revenue["Revenue"].replace("", float("nan"), inplace=True)
gme_revenue.dropna(subset=["Revenue"], inplace=True)
gme_revenue["Revenue"] = gme_revenue["Revenue"].astype(float)

# Ensure Date columns are datetime
gme_data["Date"] = pd.to_datetime(gme_data["Date"])
gme_revenue["Date"] = pd.to_datetime(gme_revenue["Date"])

# Filter data up to June 2021
gme_data_filtered = gme_data[gme_data["Date"] <= "2021-06-01"]
gme_revenue_filtered = gme_revenue[gme_revenue["Date"] <= "2021-06-01"]

# Call the make_graph function
make_graph(gme_data_filtered, gme_revenue_filtered, "GameStop")
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_11812\109047474.py:5: UserWarning:

The argument 'infer_datetime_format' is deprecated and will be removed in a future version. A strict version of it is now the defau

C:\Users\User\AppData\Local\Temp\ipykernel_11812\109047474.py:6: UserWarning:

The argument 'infer_datetime_format' is deprecated and will be removed in a future version. A strict version of it is now the defau
```





About the Authors:

Joseph Santarcangelo has a PhD in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

Azim Hirjani

Change Log

Date (YYYY-MM-DD)	Version	Changed By	Change Description